

Starting with the database layer is the perfect move. If your data isn't structured correctly, building an API or a message queue on top of it will be a nightmare. In this phase, we are ripping out the volatile "temporary memory" (your static vectors) and replacing it with a permanent, robust PostgreSQL foundation.

Here is the detailed blueprint for Phase 1, outlining exactly what to develop and how to prove it works before moving on.

1. The Database Schema Design (PostgreSQL)

You need to translate your C++ OOP attributes into relational database tables. Because we are stripping out the raw pointers and unused attributes, your tables will be remarkably clean.

Create a .sql file to define your schema. Here is the architecture you should build:

Table Name	Primary Key	Columns & Data Types	Foreign Keys & Relationships
Planets	name (VARCHAR)	x (FLOAT), y (FLOAT), z (FLOAT)	None
Travellers	id (SERIAL)	name (VARCHAR), role (VARCHAR)	role distinguishes Passenger, Astronaut, or Commander.
Tickets	ticket_id (SERIAL)	price (FLOAT), date (DATE), status (VARCHAR)	source_planet (FK -> Planets), dest_planet (FK -> Planets), traveller_id (FK -> Travellers)
SpaceTravels	travel_id (SERIAL)	date (DATE)	source_planet (FK), dest_planet (FK), astro_id (FK), commander_id (FK)
Travel_Manifest	id (SERIAL)	(Mapping table for many-to-many relationship)	travel_id (FK -> SpaceTravels), ticket_id (FK -> Tickets)

2. CMake and libpqxx Setup

Since you are building a fresh project, you need to set up your build system.

- Create a CMakeLists.txt file at the root of your project.
- Configure it to find and link the libpqxx library (the official C++ API for PostgreSQL).
- Write a basic Database Connection class. This should be a Singleton or a shared utility that holds your connection string (e.g., postgresql://user:password@localhost/spacetravel) and manages the connection pool.

3. Refactoring the C++ OOP State

This is where you surgically alter your old code to fit the new paradigm.

- **Delete the Vectors:** Completely remove static vector<Ticket> pendingTicket, Planet::globalPlanet, and Traveller::all_travellers.
- **Remove Raw Pointers:** Change Traveller *person inside your Ticket class to int traveller_id.
- **Build a Data Access Object (DAO):** Instead of your classes saving themselves, create a repository layer. For example, a TicketRepository::save(Ticket t) function that takes a Ticket object, extracts its variables, and executes the INSERT INTO Tickets... SQL command using libpqxx.
- **Hydration Methods:** Write functions that do the reverse. A TicketRepository::findByid(int id) should run a SELECT query, pull the row from PostgreSQL, and use that data to instantiate and return a C++ Ticket object.

The "End of Phase 1" Testing Checklist

At the end of this phase, you are not building the web API yet. You should build a simple, temporary test_main.cpp purely to verify your database connection.

You will know this phase is 100% complete and tested when you can successfully execute the following sequence:

- **Test 1 (Creation):** Run your C++ program to instantiate a Planet and a Traveller object, and pass them to your repository to save.
- **Test 2 (Database Verification):** Open pgAdmin or your psql terminal. Run SELECT * FROM Travellers; and visually confirm the data was written to the hard drive.
- **Test 3 (Persistence):** Close your C++ program entirely.
- **Test 4 (Retrieval):** Start your C++ program again. Write a command to fetch the user you just created. The program should successfully query the DB, recreate the Traveller object in memory, and print their name to the console.
- **Test 5 (Pricing Math):** Fetch two planets from the DB, create a Ticket object, and assert that your mathematical pricing logic ($\text{\$K} / \text{\texttt{\textbackslash text{time_to_travel}}}$) still calculates the correct price before saving the ticket.

Are you comfortable writing raw SQL queries for table creation, or would you like me to draft the exact CREATE TABLE script for PostgreSQL to get you started?